

Tyler Harris
599 Edge AI and Robotics
Module 6 TTS ASR NLP

Overview:

This lab is split into two parts. The first builds on the TTS example from the previous lab where instead of directly converting user input to text, the input is instead passed through a dictionary (free-dictionary-api), the meaning is derived, and this is then converted to text by parsing the output. The second part of the lab involves combining all three models (TTS, ASR, NLP) into a "healthcare chatbot". The pipeline for the latter part of this lab is to take user input either in the form of spoken word through a microphone or .wav file, use ASR to convert the speech to text form, use extractive NLP QA to build an answer from a provided context, and lastly use TTS to convert this response to speech which is output to either a device or a .wav file.

Part 1: TTS Dictionary

Implementation:

Building on the previous lab, the shift to outputting the dictionary meaning of user input (optionally synonyms, adjectives, etc.) is very simple with the free-dictionary-api. Instead of directly passing the input to the TTS model, the word is appended to the api url, the meaning returned, which is then passed to the TTS model. As expected the model then converts the definition to speech.

```
Enter text, or Q to quit:
> cat

--- noun ---
An animal of the family Felidae:
A person:
A strong tackle used to hoist an anchor to the cathead of a ship.
Short form of cat-o'-nine-tails.
A sturdy merchant sailing vessel (now only in "catboat").
The game of "trap and ball" (also called "cat and dog").
The pointed piece of wood that is struck in the game of tipcat.
A vagina, a vulva; the female external genitalia.
A double tripod (for holding a plate, etc.) with six feet, of which three rest on the ground, in whatever position it is placed.
A wheeled shelter, used in the Middle Ages as a siege weapon to allow assailants to approach enemy defences.

--- verb ---
To hoist (the anchor) by its ring so that it hangs at the cathead.
To flog with a cat-o'-nine-tails.
```

An output file of the TTS definition can be found under /media.

Part 2: Healthcare Chatbot

Implementation:

The purpose of this portion of the lab is to combine all three of the applications demonstrated in the lectures (TTS, ASR, NLP). First speech is input to the system through an input device or a

.wav file representing spoken input. The ASR class from the jetson-voice repository is used with the quartznet model by default in order to convert the input to plain text. This plain text is then passed to the NLP QA class from the repository which uses a context provided in the code to extract the answer from the provided data based on the user query. After the answer is extracted from the text it is passed to the TTS class in order to output the response as speech either to an output device or .wav file.

Initializing models:

```
asr = ASR(args.asr_model)
qa_model = QuestionAnswer(args.model)
tts = TTS("fastpitch_hifigan")
```

Provided Context:

```
medical_context = (
    "Type 2 diabetes is a chronic condition affecting how the body metabolizes glucose. "
    "The main symptoms of type two diabetes include increased thirst, frequent urination, fatigue, and blurred vision. "
    "Treatment focuses on diet, exercise, and blood sugar monitoring. "
    "During a sudden asthma attack, standard clinical protocol states that patients should sit upright, "
    "use a rescue inhaler immediately, and seek emergency care if symptoms do not improve. "
    "Avoid lying down as it restricts airway capacity. "
    "The primary goal of a medical triage chatbot is to assess the severity of symptoms to route patients "
    "to the correct care level. It provides automated guidance based on established clinical assessment protocols."
)
```

Read user input (ASR):

```
print(f"\nReading and transcribing audio question file: {args.wav}")
stream = AudioInput(wav=args.wav, sample_rate=asr.sample_rate, chunk_size=asr.chunk_size)
```

Get QA Result (NLP):

```
query = {
    'context': medical_context,
    'question': user_question
}

qa_results = qa_model(query, top_k=args.top_k)
```

Convert QA Result to Speech (TTS):

```
for run in range(args.warmup + 1):
    start = time.perf_counter()
    audio = tts(response_text)
    stop = time.perf_counter()

    latency = stop - start
    duration = audio.shape[0] / tts.sample_rate
```

Results:

The system works relatively well given the small dictionary size. There are two provided examples below.

Prompt: How can I manage diabetes?

Response: < put response here>

The .wav file of the response can be found under /media

Accuracy: In this case the program outputs the common symptoms of diabetes despite treatments being directly provided in the context. Initially I suspected this may be because of the usage of the word “manage” instead of treat and the NLP model chose the answers from the dictionary that directly contained diabetes. However after testing with the prompt “How can I treat diabetes” it provided the same response.

```
"Type 2 diabetes is a chronic condition affecting how the body metabolizes glucose. "  
"The main symptoms of type two diabetes include increased thirst, frequent urination, fatigue, and blurred vision. "  
"Treatment focuses on diet, exercise, and blood sugar monitoring. "
```

Prompt: I am having an asthma attack

Response: < response here>

The .wav file of the response can be found under /media

Accuracy: This was a good example of the NLP proving more accurate when finding an answer. It pulled the answer that is defined in the context under in the case of an asthma attack and correctly pulled the treatment. This is a positive example however it does lead to some curiosity as to why the diabetes prompt did not yield similar results.

```
"During a sudden asthma attack, standard clinical protocol states that patients should sit upright, "  
"use a rescue inhaler immediately, and seek emergency care if symptoms do not improve. "  
"Avoid lying down as it restricts airway capacity. "
```